

Rapport Technique Approfondi : Système de Gestion Morphologique Arabe

Option : 1ING GLSI

Année Universitaire 2025-2026

1 Introduction

L'objectif de ce projet est de concevoir un moteur de recherche et de génération morphologique pour la langue arabe, en exploitant le modèle "Racine-Schème". Ce document détaille les choix algorithmiques, les structures de données complexes mises en œuvre (Arbres AVL, Tables de hachage multiples) ainsi qu'une analyse exhaustive de leurs performances.

2 Architecture des Structures de Données

2.1 Arbre AVL : Gestion Dynamique du Lexique

Le choix de l'**Arbre AVL** (`AVLTree.h`) répond à la nécessité d'un dictionnaire de racines évolutif.

- **Équilibrage Strict** : Contrairement aux arbres binaires classiques, l'AVL garantit que la différence de hauteur entre deux sous-arbres ne dépasse jamais 1. Cela évite la dégénérescence en liste chaînée dans le cas de racines insérées par ordre alphabétique.
- **Stockage des Familles** : Chaque nœud contient une structure `vector<DerivedWord>`, permettant de lier une racine à l'ensemble des mots détectés dans le corpus, avec leurs fréquences respectives.

2.2 Table de Hachage à Multiples Index (Schemes)

Pour les schèmes (`schemeHashTable.h`), nous avons implémenté une table de hachage à **chaînage séparé** utilisant trois indexations distinctes pour optimiser les performances de recherche selon le contexte :

1. **Index par Nom** : Pour une récupération directe (ex : "maF3ouL").
2. **Index par Pattern** : Pour vérifier si une structure morphologique existe déjà.
3. **Index par Longueur** : Un tableau de listes (`lengthTable[30]`) qui permet au moteur d'analyse de ne tester que les schèmes compatibles avec la taille du mot traité, réduisant drastiquement le nombre de comparaisons.

3 Description des Algorithmes et Fonctionnalités

3.1 Génération Morphologique (Forward)

L'algorithme de génération est une fonction de transformation $f(R, S) \rightarrow M$.

- **Processus** : Il décompose le schème en caractères UTF-8. À chaque rencontre des marqueurs '1', '2' ou '3', il injecte le radical correspondant de la racine.
- **Traitement de la Shadda** : Une étape de prétraitement (`expandShadda`) est appliquée. Si une Shadda est détectée, la lettre la précédant est dupliquée pour l'analyse, garantissant que le mot généré respecte les règles de gémination arabe.

3.2 Analyseur de Corpus et Apprentissage

L'analyseur (`CorpusAnalyzer.h`) parcourt des fichiers textes. Pour chaque mot, il tente de "deviner" la racine via les schèmes connus.

- **Mode Apprentissage** : Si une structure correspond à un schème mais que la racine est absente de l'AVL, l'algorithme insère cette nouvelle racine. C'est une fonctionnalité d'auto-enrichissement du dictionnaire.

4 Analyse Complète des Complexités

Nous distinguons ici la complexité temporelle (vitesse) et la complexité spatiale (occupation mémoire).

4.1 Complexité Temporelle (Time Complexity)

Opération	Meilleur Cas	Pire Cas	Justification
Recherche Racine (AVL)	$O(1)$	$O(\log n)$	Arbre toujours équilibré.
Insertion Racine (AVL)	$O(\log n)$	$O(\log n)$	Inclut le temps de rééquilibrage.
Recherche Schème	$O(1)$	$O(1 + \alpha)$	α est le facteur de charge.
Extraction de Racine	$O(1)$	$O(K_L \cdot L)$	K_L : nb schèmes de longueur L .
Visualisation de l'Arbre	$O(n)$	$O(n)$	Parcours complet pour le dessin (Qt).
Analyse Corpus (N mots)	$O(N \cdot \log n)$	$O(N \cdot K_L \cdot \log n)$	Dépend de la diversité des schèmes.

4.2 Complexité Spatiale (Space Complexity)

- **Arbre AVL** : $O(n + D)$ où n est le nombre de racines et D le nombre total de mots dérivés stockés. Chaque nœud utilise des pointeurs intelligents (`unique_ptr`), optimisant la libération mémoire.
- **Table de Hachage** : $O(S)$ où S est le nombre de schèmes. L'utilisation de listes chaînées pour les collisions minimise la mémoire gaspillée par rapport à un adressage ouvert.

5 Difficultés Rencontrées et Solutions

1. **Sémantique de l'UTF-8** : La gestion des caractères arabes (2 octets par caractère) rend les fonctions standards comme `std::string::size()` obsolètes. Nous

avons dû implémenter `StringUtils::splitUTF8` pour itérer correctement sur les glyphes.

2. **Rendu Graphique :** L’affichage d’un Arbre AVL avec des milliers de nœuds dans l’interface Qt a nécessité l’implémentation d’un système de **Zoom et Pan** (`ZoomableView`) et d’un calcul de coordonnées récursif pour éviter le chevauchement des nœuds.

6 Conclusion

Le système atteint un haut degré d’efficacité grâce à la spécialisation des structures de données. L’indexation multiple des schèmes et l’équilibrage logarithmique des racines permettent de traiter des volumes de données importants sans latence perceptible pour l’utilisateur.